ЛР2. Система автоматической категоризации контента

1/ Цель работы

Легенда:
Представьте, что вы — NLP-инженер в крупном новостном издании или аналитическом агентстве. Ежедневно поступают тысячи текстов: новости, посты в блогах, технические статьи. Вручную сортировать их по рубрикам (Спорт, Политика, Технологии, Медицина) невозможно.

Цель
Разработать прототип системы, которая автоматически определяет тематику текста. Вам предстоит пройти полный цикл Data Science проекта: от предобработки сырых текстов до создания веб-приложения, где пользователь может проверить работу модели и исправить её ошибки.

2/ Теория

Фундаментальная задача NLP: научить компьютер "понимать" человеческий язык. При работе с алгоритмами ML мы оперируем не токенами, а числами, векторами и матрицами. Важнейшим этапом решения задач NLP является процесс **векторизации текстовых данных**.

В задачах, связанных с искусственным интеллектом, данные могут быть:

- структурированные (таблицы, DataFrame, изображения)
- неструктурированные (текст, звук)

Структурированные данные — данные, которые можно представить в виде списка (множества) векторов. Неструктурированные данные требуется привести к виду, в котором они могут быть представлены в виде списка векторов.

Каждый элемент вектора, который характеризует объект данных, называется **признаком** (feature). В виду этого процесс векторизации является **процессом извлечения признаков** (feature extraction).

> ⓘ **Info**
>
> **Цель векторизации** в современном NLP — создание многомерного семантического пространства, в котором геометрия отражает смысловые отношения между языковыми единицами

Два основных подхода к векторизации токенов:

- **частотные модели** (вектор слова или документа можно определить через частоту встречаемости слов), легко реализуемы и интерпретируемы, но генерируют векторы разреженные векторы, размерность которых равна размеру словаря
- **предиктивные модели** (плотные эмбеддинги), обучаются предсказывать слово на основе его окружения (контекста) или наоборот (применяются нейронные сети)

2.1/ Bag-of-Words

Относится к частотным методам. Основная идея заключается в представлении каждого документа как неупорядоченного набора слов, где важна исключительно частота их появления (порядок слов игнорируются).

Процесс создания BoW-представления для корпуса документов выглядит так:

- документы разбиваются на отдельные слова (токены)
- формируется словарь всех уникальных токенов из корпуса
- каждый документ представляется в виде вектора, размерность которого равна размеру словаря. Каждое значение в векторе соответствует количеству вхождений (частоте) слова из словаря в данный документ.

Два документа:

```
Документ 1: "Мама мыла раму, раму мыла мама."
Документ 2: "Мама сегодня мыла окно."
```

Словарь (после приведения к нижнему регистру и удаления знаков препинания): `["мама", "мыла", "раму", "сегодня", "окно"]`.
BoW-векторы для документов будут следующими:

```
Документ 1: [2, 2, 2, 0, 0]
Документ 2: [1, 1, 0, 1, 1]
```

Важные недостаток: потеря порядка слов и контекста ("собака укусила человека" и "человек укусил собаку") - **общая проблема всех подходов к векторизации на уровне токенов.**

2.2/ Term Frequency-Inverse Document Frequency (TF-IDF)

Идея: вес слова в документе должен быть пропорционален его частоте в этом документе (локальный компонент) и обратно пропорционален его частоте во всем корпусе документов (глобальный компонент). Слова, которые часто встречаются в конкретном документе, но редко в остальных, получают наибольший вес.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

Где N — число документов, $DF(t)$ — число документов, содержащих терм t.

2.3/ Word2Vec

Относится к **предиктивным моделям** векторизации. Частотные модели не улавливают семантическую связь между словами.

Основная идея в том, что слова, которые встречаются в схожих контекстах, имеют **схожие значения**. Например, слова "кофе" и "чай" часто встречаются в контексте слов "чашка", "горячий", "сахар", "пить". Следовательно, их векторные представления **должны быть близки**. Плотные векторы, получаемые с применением предиктивных моделей, имеют значительно

меньшую размерность (обычно от 50 до 300). Эти векторы обучаются на огромных корпусах текстов так, что слова, встречающиеся в похожих контекстах, имеют близкие векторы.

Магия: Векторы обладают линейной структурой. Например: `vector("король") - vector("мужчина") + vector("женщина") ≈ vector("королева")`.

Архитектуры обучения:

- **CBOW (Continuous Bag-of-Words):** Предсказать слово по его контексту.
- **Skip-gram:** Предсказать контекст по слову (обычно лучше работает для редких слов).

2.4/ Doc2Vec (Paragraph Vector)

Идея. Аналог Word2Vec, но на уровне документов. Добавляется еще один вектор (ID документа), который "запоминает" отсутствующую в контексте тему всего документа. Позволяет получать эмбеддинги для текстов разной длины.

2.5/ Алгоритмы классификации

1. **SVM (Support Vector Machine):** Ищет гиперплоскость, максимизирующую зазор (margin) между классами. Эффективен в пространствах высокой размерности (как текст).
2. **Random Forest:** Ансамбль решающих деревьев. Устойчив к переобучению, работает с нелинейными зависимостями.
3. **Gradient Boosting (например, XGBoost/LightGBM):** Последовательное построение деревьев, исправляющих ошибки предыдущих.

2.6/ Метрики оценки

- **Precision (Точность):** Какая доля найденных положительных объектов является действительно положительными.
- **Recall (Полнота):** Какая доля реальных положительных объектов была найдена.
- **F1-score:** Гармоническое среднее Precision и Recall.

3/ Ход работы

Шаг 1/ Подготовка окружения и данных

Установите необходимые библиотеки:

```
pip install scikit-learn gensim streamlit pandas numpy matplotlib seaborn nltk
```

Для воспроизводимости используем датасет `20 Newsgroups` из `sklearn`. Он содержит ~18 000 постов в 20 категориях. Для ускорения обучения возьмем подмножество из 4-5 категорий (например, связанные с технологиями или спортом).

Пример кода

```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_20newsgroups
```

```python
from sklearn.model_selection import train_test_split

# 1. Загрузка данных
categories = ['sci.space', 'sci.med', 'comp.graphics', 'rec.sport.baseball']
newsgroups = fetch_20newsgroups(subset='all', categories=categories, remove=
('headers', 'footers', 'quotes'))

# 2. Создание DataFrame
df = pd.DataFrame({
    'text': newsgroups.data,
    'target': newsgroups.target,
    'target_name': [newsgroups.target_names[i] for i in newsgroups.target]
})

# 3. Разделение на Train/Test (Стратифицированное!)
X_train, X_test, y_train, y_test = train_test_split(
    df['text'],
    df['target'],
    test_size=0.2,
    random_state=42,
    stratify=df['target']
)

print(f"Размер выборки: {len(df)}, Классы: {df['target_name'].unique()}")
```

Шаг 2/ Предобработка и векторизация

Создадим функцию для лемматизации (используем `nltk` или простую токенизацию для скорости). В реальном проекте этот шаг критичен.

> **Задание**
>
> Реализуйте три способа векторизации и сравните размерность получаемых матриц.

Пример кода

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from gensim.models import Word2Vec, Doc2Vec
from gensim.utils import simple_preprocess
from sklearn.base import BaseEstimator, TransformerMixin
import nltk
# nltk.download('stopwords') # Раскомментировать при первом запуске
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

# --- Векторизатор 1: TF-IDF (Scikit-Learn) ---
tfidf_vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
```

```
    ngram_range=(1, 2) # Учитываем биграммы
)

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

# --- Векторизатор 2: Word2Vec (Gensim + Mean Pooling) ---
# Word2Vec требует список списков токенов
def preprocess_text(text):
    tokens = simple_preprocess(text)
    return [t for t in tokens if t not in stop_words]

train_tokens = [preprocess_text(t) for t in X_train]
test_tokens = [preprocess_text(t) for t in X_test]

# Обучение Word2Vec на тренировочных данных
w2v_model = Word2Vec(sentences=train_tokens, vector_size=100, window=5, min_count=1, workers=4)

# Функция для превращения документа в вектор (усреднение векторов слов)
def document_vector(doc, model):
    doc_vec = np.array([model.wv[word] for word in doc if word in model.wv])
    return doc_vec.mean(axis=0) if doc_vec.any() else np.zeros(model.vector_size)

X_train_w2v = np.array([document_vector(doc, w2v_model) for doc in train_tokens])
X_test_w2v = np.array([document_vector(doc, w2v_model) for doc in test_tokens])

print(f"TF-IDF shape: {X_train_tfidf.shape}, W2V shape: {X_train_w2v.shape}")
```

Шаг 3/ Обучение и сравнение классификаторов

Используем `Pipeline` для TF-IDF (так как он встроен в sklearn), а для Word2Vec обучаем модели отдельно, так как векторизация уже выполнена.

> **Задание**
>
> Обучите SVM, Random Forest и Gradient Boosting. Заполните таблицу метрик.

Пример кода

```
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import classification_report, accuracy_score

models = {
    'SVM (TF-IDF)': LinearSVC(random_state=42),
    'RF (TF-IDF)': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM (W2V)': LinearSVC(random_state=42), # Используем те же классификаторы для честности
    'RF (W2V)': RandomForestClassifier(n_estimators=100, random_state=42)
```

```
}

# Данные для SVM/RF на TF-IDF
data_tfidf = {'X_train': X_train_tfidf, 'X_test': X_test_tfidf}
# Данные для SVM/RF на W2V
data_w2v = {'X_train': X_train_w2v, 'X_test': X_test_w2v}

results = []

# Обучение на TF-IDF
for name, model in [('SVM (TF-IDF)', models['SVM (TF-IDF)']), ('RF (TF-IDF)', models['RF (TF-IDF)'])]:
    model.fit(data_tfidf['X_train'], y_train)
    preds = model.predict(data_tfidf['X_test'])
    acc = accuracy_score(y_test, preds)
    f1 = classification_report(y_test, preds, output_dict=True)['weighted avg']['f1-score']
    results.append({'Model': name, 'Accuracy': acc, 'F1-Score': f1, 'Type': 'TF-IDF'})

# Обучение на W2V
for name, model in [('SVM (W2V)', models['SVM (W2V)']), ('RF (W2V)', models['RF (W2V)'])]:
    model.fit(data_w2v['X_train'], y_train)
    preds = model.predict(data_w2v['X_test'])
    acc = accuracy_score(y_test, preds)
    f1 = classification_report(y_test, preds, output_dict=True)['weighted avg']['f1-score']
    results.append({'Model': name, 'Accuracy': acc, 'F1-Score': f1, 'Type': 'Word2Vec'})

results_df = pd.DataFrame(results)
print(results_df.sort_values(by='F1-Score', ascending=False))
```

Шаг 4/ Разработка интерфейса (Streamlit)

Создайте файл `app.py`. Интерфейс должен позволять ввести текст, получить предсказание лучшей модели и **исправить** его, если оно неверно. Ошибки должны сохраняться в `feedback.csv`.

Пример кода

```
# app.py
import streamlit as st
import pandas as pd
import pickle
from sklearn.feature_extraction.text import TfidfVectorizer
import os

# Загрузка модели и векторизатора (предполагается, что вы сохранили лучшую модель в Шаге 3)
# Для примера кода создадим "заглушку", в реальности используйте pickle.dump/load
```

```python
# @st.cache_resource
# def load_model():
#     ...

st.title("📰 Классификатор новостей")
st.write("Введите текст статьи, чтобы определить категорию.")

# Ввод текста
user_input = st.text_area("Текст статьи", height=200)

if st.button("Классифицировать"):
    if not user_input:
        st.warning("Введите текст!")
    else:
        # Здесь должна быть логика предсказания
        # prediction = model.predict(vectorizer.transform([user_input]))[0]
        # Для демо используем случайный выбор из категорий
        categories = ['sci.space', 'sci.med', 'comp.graphics', 'rec.sport.baseball']
        predicted_class = categories[hash(user_input) % len(categories)]

        st.success(f"Предсказанная категория: **{predicted_class}**")

        # Блок коррекции (Human-in-the-loop)
        with st.form("feedback_form"):
            st.write("Если предсказание неверно, укажите правильный класс:")
            true_class = st.selectbox("Верный класс", categories)
            submitted = st.form_submit_button("Отправить исправление")

            if submitted:
                feedback_data = {
                    'text': user_input[:100] + "...", # Сохраняем начало для краткости
                    'predicted': predicted_class,
                    'corrected': true_class,
                    'is_error': predicted_class != true_class
                }

                # Сохранение в CSV
                df_feedback = pd.DataFrame([feedback_data])
                if not os.path.exists("feedback.csv"):
                    df_feedback.to_csv("feedback.csv", index=False)
                else:
                    df_feedback.to_csv("feedback.csv", mode='a', header=False, index=False)

                st.info("Спасибо! Ваше исправление сохранено в feedback.csv")

# Просмотр логов (для преподавателя)
if os.path.exists("feedback.csv"):
    with st.expander("Посмотреть логи исправлений"):
        st.dataframe(pd.read_csv("feedback.csv"))
```

Запуск приложения:

```
streamlit run app.py
```

Форма отчета

1. Файл Jupyter Notebook (или Google Colab) с ходом работы (векторизация, обучение, оценка)
2. Файл `app.py` с приложением на Streamlit